Section 8

Strings

DF

Direction Flag:

1 -> operation for the DI and SI (increment)

2 -> operation for the DI and SI (Decrement)

DF

CLD -> DF = 0

STD -> DF = 1

String Instructions

MOVSB/MOVSW

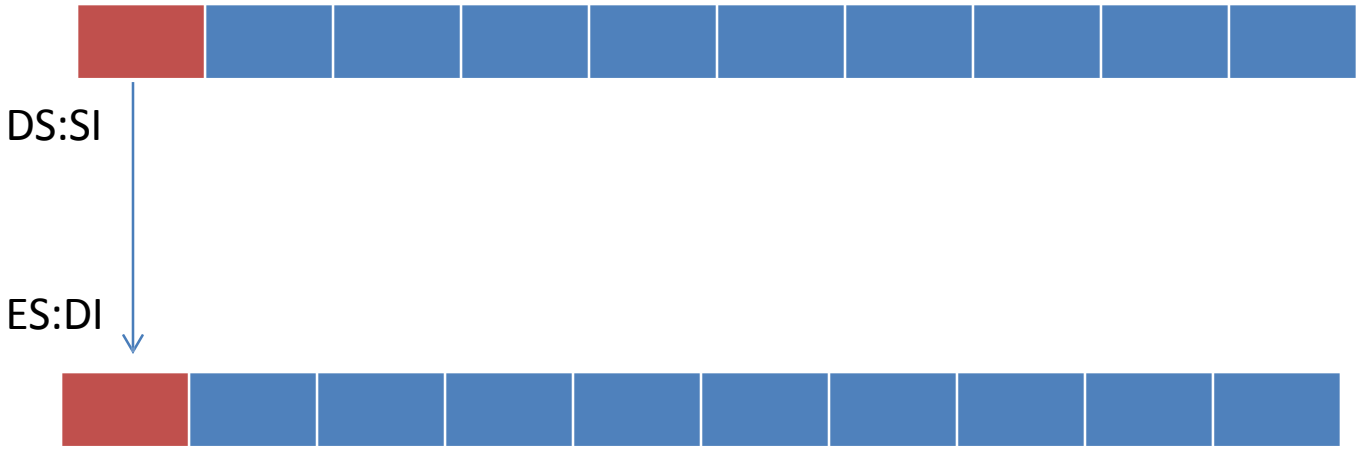
- **MOVSB**

The MOVSB (move string, byte) instruction fetches the byte at address DS:SI, stores it at address ES:DI and then increments or decrements the SI and DI registers by one.

- **MOVSW**

The MOVSW (move string, word) instruction fetches the word at address DS:SI, stores it at address ES:DI and then increments or decrements SI and DI by two.

MOVSB



MOVSB

DS:SI

Then increments SI and DI (IF DF==0)

ES:DI

N.B. SI and DI are decremented if DF==1

Example MOVSB

Copy all elements in array1 to array2

```
.model small
.stack 128
.data

   Array1 db 10 dup('m')
   Array2 db 10 dup(?)

.code
main proc far
; set segment registers:
   mov ax, data
   mov ds, ax
   mov es, ax
   mov DI, offset Array2
   mov SI, offset Array1
```

```
MOV CX, 10
CLD
next:      MOVSB
           LOOP next
 mov ah, 4ch.
 int 21h
main endp
end main
```

REP

REP: Allows the execution to be repeated CX times.

Example MOVSB using REP

Copy all elements in array1 to array2

```
.model small
.stack 128
.data

   Array1 db 10 dup('m')
   Array2 db 10 dup(?)

.code
main proc far
; set segment registers:
   mov ax, data
   mov ds, ax
   mov es, ax
   mov DI, offset Array2
   mov SI, offset Array1
```

```
MOV CX, 10

CLD
REP MOVSB

 mov ah, 4ch.
 int 21h
main endp
end main
```

CMPSB/CMPSW

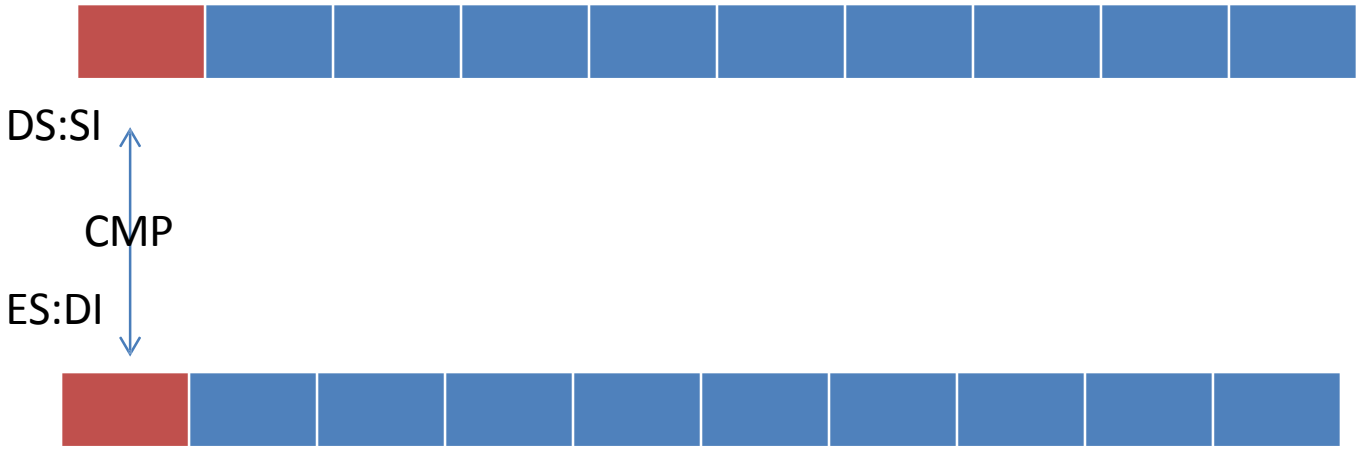
- **CMPSB**

This instruction is used to compare arrays of bytes. It compares the byte addressed by DS:[SI] to the byte addressed by ES:[DI]. Then, SI and DI are either incremented or decremented by 1 depending on the Direction Flag.

- **CMPSW**

This instruction is used to compare arrays of words. It compares the word addressed by DS:[SI] to the word addressed by ES:[DI]. Then, SI and DI are either incremented or decremented by 2 depending on the Direction Flag.

CMPSB



CMPSB

DS:SI

Then increments SI and DI (IF DF==0)

ES:DI

N.B. SI and DI are decremented if DF==1

Example CMPSB

Compare all elements in array1 to array2; if the 2 arrays are identical
The Equal variable should be one, if not Equal variable should be 0

```
.model small
.stack 128
.data

   Array1 db 10 dup('m')
   Array2 db 10 dup('m')
   Equal db 0

.code
main proc far
; set segment registers:
   mov ax, data
   mov ds, ax
   mov es, ax
   mov DI, offset Array2
   mov SI, offset Array1
```

```
MOV CX, 10
CLD

Next:  CMPSB
       jne skip
       loop Next

Skip:
      cmp cl,0
      jne end
      mov  Equal ,1
end:
mov ah, 4ch
int 21h
main endp
end main
```

REPE/REPZ
REPNE/REPNZ

REPE/REPZ : The string instruction will repeat execution as long as CX is not equal to 0 and also the result of comparison sets the ZF to 1, i.e. the two compared operands are equal.

REPNE/REPNZ : The string instruction will repeat execution as long as CX is not equal to 0 and also the result of comparison sets the ZF to 0, i.e. the two compared operands are not equal.

Example CMPSB and REPE

Compare all elements in array1 to array2; if the 2 arrays are identical
The Equal variable should be one, if not Equal variable should be 0

```
.model small
.stack 128
.data

   Array1 db 10 dup('m')
   Array2 db 10 dup('m')
   Equal db 0

.code
main proc far
; set segment registers:
   mov ax, data
   mov ds, ax
   mov es, ax
   mov DI, offset Array2
   mov SI, offset Array1
```

```
MOV CX, 10
CLD

repe  CMPSB

jne end
mov  Equal ,1
end:
mov ah, 4ch
int 21h
main endp
end main
```

SCASB/ SCASW

SCASB/SCASW :

The scan string instructions are useful in searching for a particular value or a character in a string. These instructions are used to compare arrays to a byte or word. Instructions only require a destination string (pointed at by ES:DI). The source operand is the value in the AL (SCASB), AX (SCASW). Then increments (or decrements) DI by one or two.

SCASB

SCASB

Then increments SI and DI (IF DF==0)

N.B. SI and DI are decremented if DF==1

Example SCASB

Search of Letter in array1; if found the Found variable should be one, if not Found variable should be 0

```
.model small
.stack 128
.data

   Array1 db "HELLO"
   Letter db 'z'
   Found db 0

.code
main proc far
; set segment registers:
   mov ax, data
   mov ds, ax
   mov es, ax
   mov DI, offset Array1
   mov al , Letter
```

```
MOV CX, 5
CLD

Loop1: scasb
          je skip
          loop loop1


Skip:
        cmp cx,0
         je end
        mov found , 1

end:

mov ah, 4ch
int 21h
main endp
end main
```

REPE/REPZ
REPNE/REPNZ

REPE/REPZ : The string instruction will repeat execution as long as CX is not equal to 0 and also the result of comparison sets the ZF to 1, i.e. the two compared operands are equal.

REPNE/REPNZ : The string instruction will repeat execution as long as CX is not equal to 0 and also the result of comparison sets the ZF to 0, i.e. the two compared operands are not equal.

Example SCASB and REPNE

Search of Letter in array1; if found the Found variable should be one, if not Found variable should be 0

```
.model small
.stack 128
.data

   Array1 db "HELLO"
   Letter db 'z'
   Found db 0

.code
main proc far
; set segment registers:
   mov ax, data
   mov ds, ax
   mov es, ax
   mov DI, offset Array1
   mov al , Letter
```

```
MOV CX, 5
CLD

repne scasb

jne end
mov found , 1

end:

mov ah, 4ch
int 21h
main endp
end main
```

STOSB/STOSW

STOSB/STOSW instructions store the value in the accumulator (AL/AX) into the corresponding locations pointed by ES:DI and then increments (or decrements) DI by one or two.

STOSB

STOSB

Then increments SI and DI (IF DF==0)

N.B. SI and DI are decremented if DF==1

Example STOSB

Copy the letter in Letter variable into all elements of array1

```
.model small
.stack 128
.data

   Array1 db "HELLO"
   Letter db 'K'


.code
main proc far
; set segment registers:
   mov ax, data
   mov ds, ax
   mov es, ax
   mov DI, offset Array1
   mov al , Letter
```

```
MOV CX, 5
CLD

loop1: stosb
          loop loop1

mov ah, 4ch
int 21h
main endp
end main
```

Example STOSB and REP

Copy the letter in Letter variable into all elements of array1

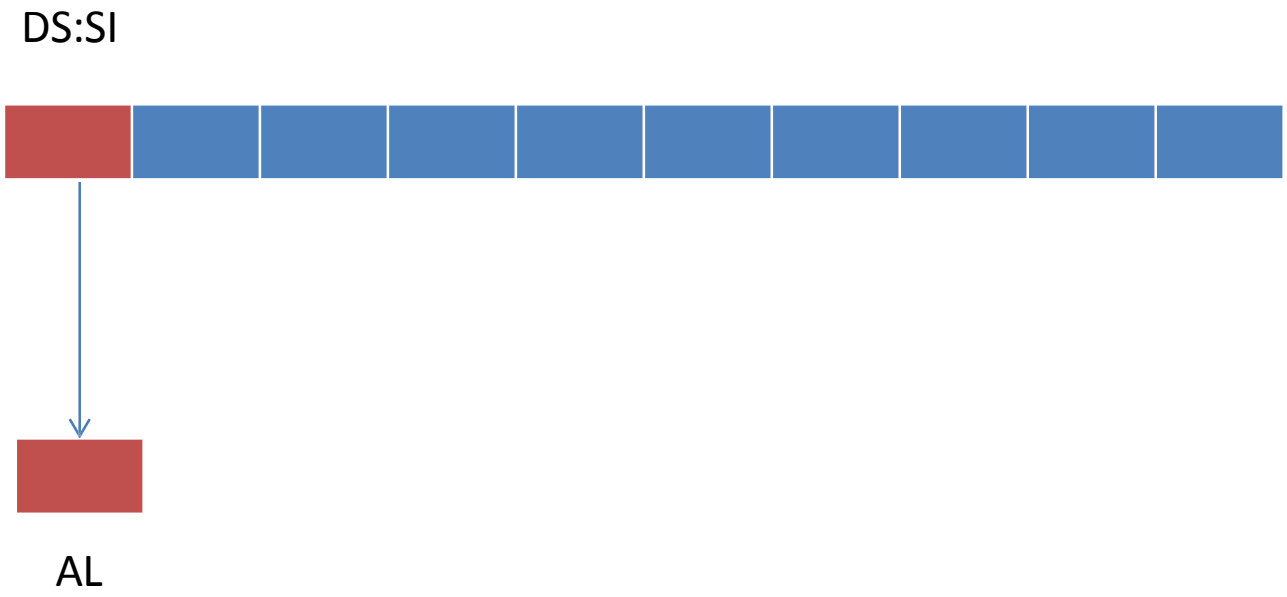
```
.model small
.stack 128
.data

   Array1 db "HELLO"
   Letter db 'K'



.code
main proc far
; set segment registers:
   mov ax, data
   mov ds, ax
   mov es, ax
   mov DI, offset Array1
   mov al , Letter
```

```
MOV CX, 5
CLD

REP stosb

mov ah, 4ch
int 21h
main endp
end main
```

LODSB/LODSW

LODSB/LODSW instructions load the value from the corresponding locations pointed by DS:SI into the accumulator (AL/AX) and then increments (or decrements) SI by one or two.

LODSB



LODSB

DS:SI

Then increments SI and DI (IF DF==0)

N.B. SI and DI are decremented if DF==1

Uppercase to Lowercase using LODSB and STOSB

```
.model small
.stack 128
.data

 Array1 db "HELLO"
 Array2 db 5 dup(?)

.code
main proc  far
    mov ax, data
    mov ds, ax
    mov es, ax
    mov  SI,offset  Array1
    mov DI,offset Array2
    MOV CX, 5
```

```
CLD

loop1: LODSB
            cmp al,'A'
            jb loop2
            cmp al,'Z'
            ja loop2
            add al,20H
          loop2:   stosb
            loop loop1


    mov ah, 4ch
    int 21h
main endp
end main
```

Repeat compatibility

REP -> MOVS, STOS

REPE/REPZ, REPNE/REPNZ -> CMPS, SCAS

Thanks